

TBI TECHNOLOGY BUSINESS INCUBATOR

AI – Assisted Full Stack Web Development

Week 4 – Frontend & Backend Connection Report

Project: Krishi AI Advisor

Intern Name: Aniket Dobriyal

Intern ID: TBI-26100505

GitHub Repository: <https://github.com/aniketdobriyal/Krishi-AI-Advisor>

Technology Stack: React 19, Vite 8, Express.js, Node.js, Axios, Tailwind CSS, Gemini API

1. Backend Integration Summary

During Week 4, the Krishi AI Advisor prototype was upgraded from a static frontend to a full-stack application. An Express.js backend following the MVC architecture was implemented with REST APIs for crops, diseases, pests, chat, history, search and configuration. Axios was integrated into the React frontend to replace hardcoded data with live API calls. CORS, centralized error handling, and environment-variable based Gemini API configuration were also implemented.

2. Frontend Dashboard Connected to Backend

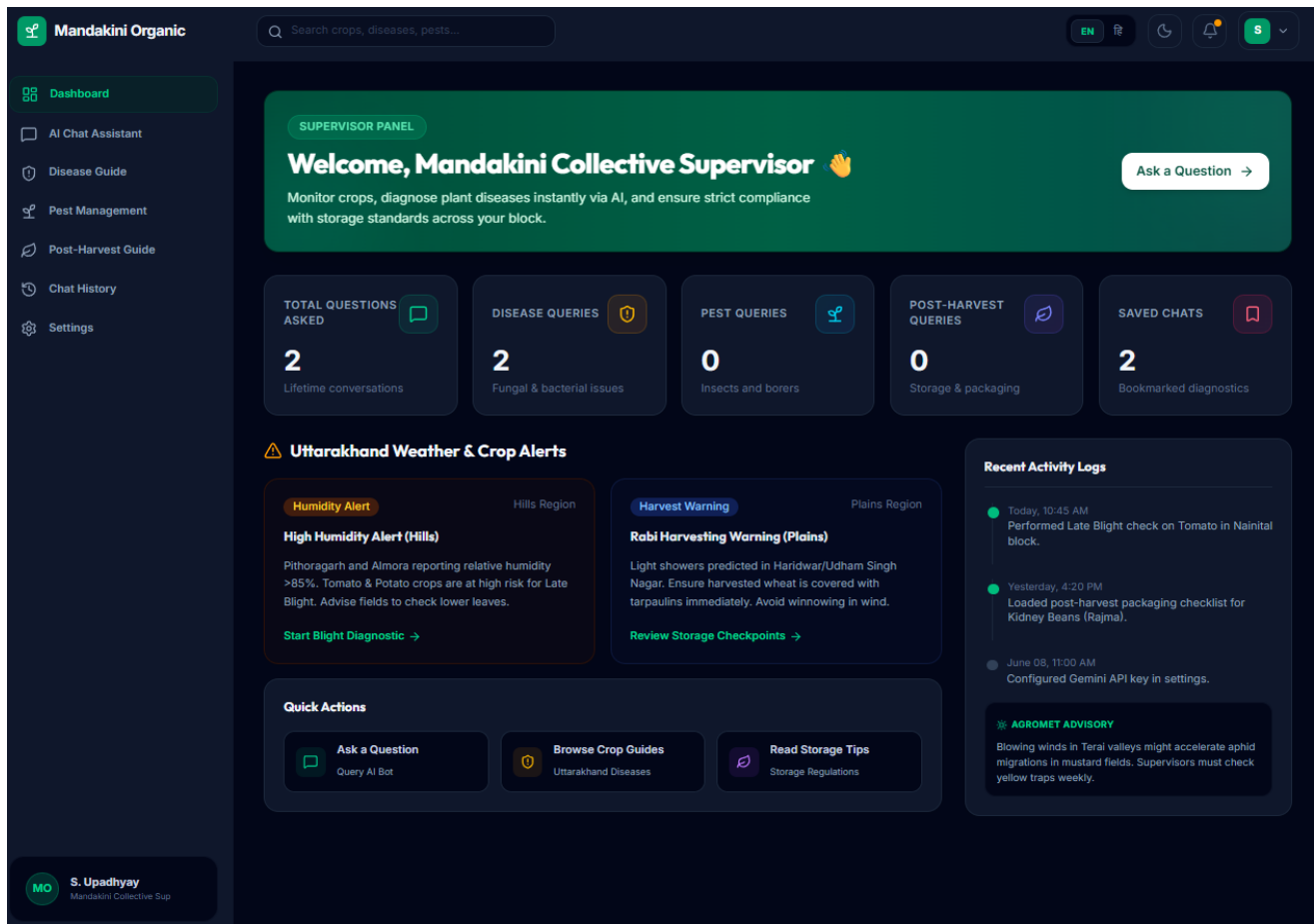


Figure 1. Dashboard displaying live crop advisory data retrieved from the Express backend.

The dashboard loads application data through backend REST APIs instead of static frontend data, demonstrating successful frontend-backend integration.

3. AI Chat Integration

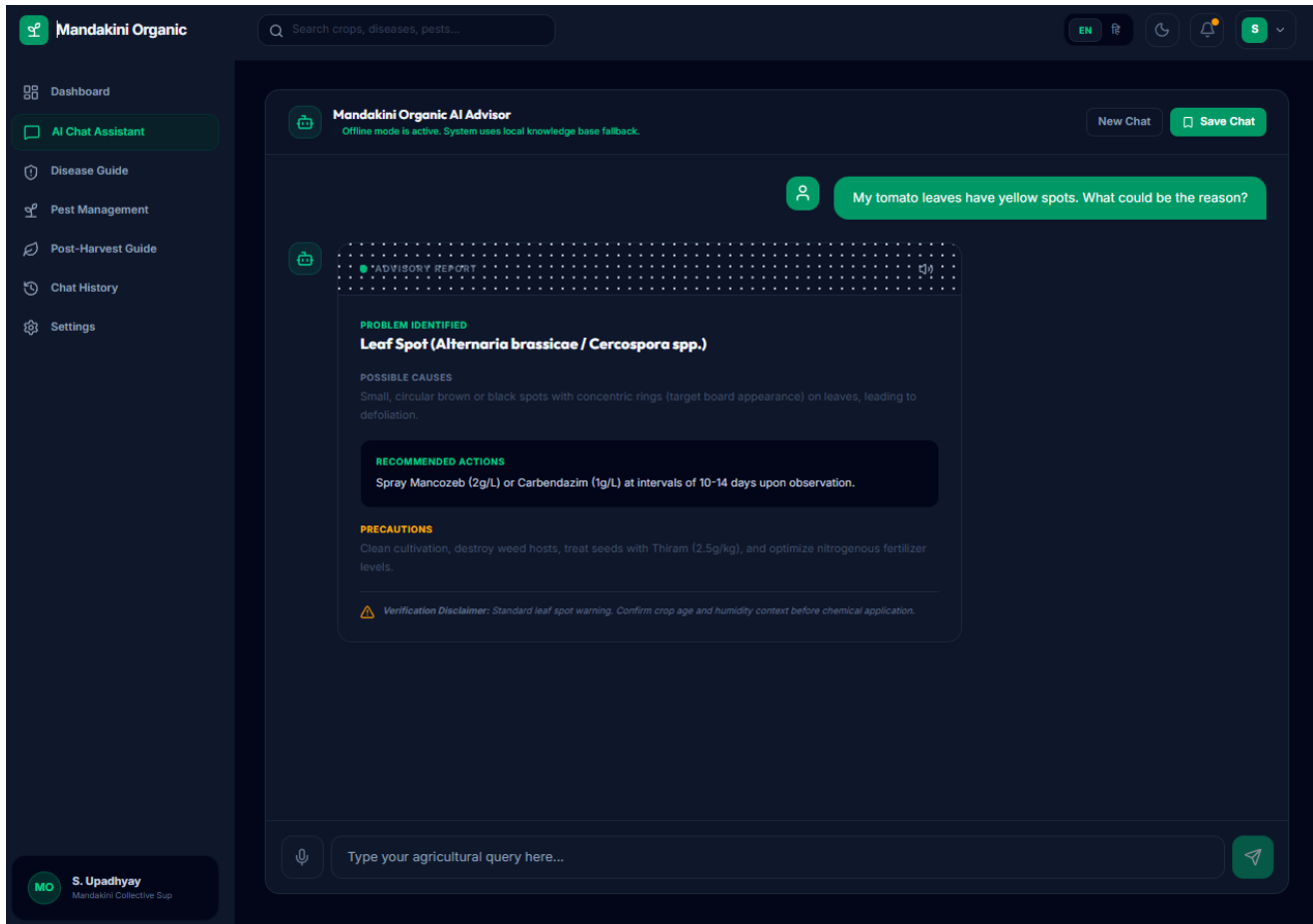


Figure 2. AI Chat Assistant communicating with the Express backend.

The chatbot sends crop advisory requests to the backend using Axios. The Express server processes requests and returns structured responses using either Gemini or the offline fallback engine.

4. REST API Communication Verification

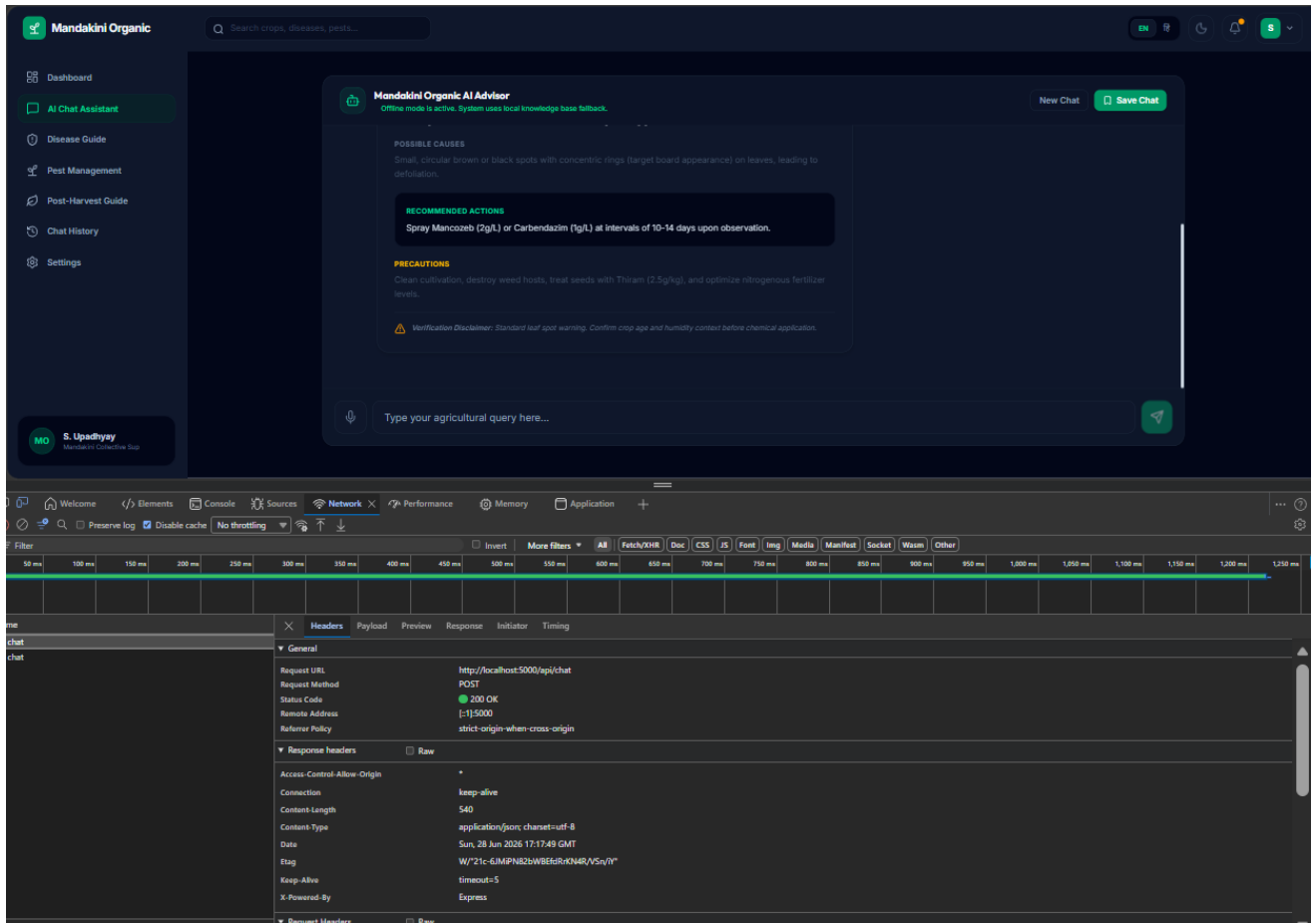


Figure 3. Chrome DevTools showing a successful POST /api/chat request (HTTP 200 OK).

The Network panel confirms successful communication between the React frontend and Express backend. The POST request returns HTTP 200 OK, verifying live API connectivity.

5. Implemented REST API Endpoints

Method	Endpoint
GET	/api/crops
GET	/api/diseases
GET	/api/diseases/:id
GET	/api/pests
GET	/api/post-harvest
POST	/api/chat
GET	/api/history
POST	/api/history
DELETE	/api/history/:id
GET	/api/search
GET	/api/config

6. Conclusion

The Week 4 objectives were successfully completed. The React frontend is fully connected to the Express.js backend through REST APIs. Backend endpoints were validated using Postman, while frontend communication was verified using Chrome DevTools with successful HTTP 200 responses.